

SSAFY

Javascript

Basic syntax 01

Python



목 차

데이터 타입

- 원시 자료형

연산자

조건문

반복문

- for ... in
- for ... of
- for ...in 과 for ... of

목 차

함수

- 함수 정의
- 매개변수
- Spread syntax
- 화살표 함수 표현식

참고

- NaN 예시
- null & undefined
- 화살표 함수 심화

장바구니 총액을 계산하고, 특정 금액이 넘으면 '무료 배송'을 적용하는 기능은 어떻게 만들까요?

1. 상품 정보를 **데이터**로 담고, **연산자**로 가격을 계산한 뒤, **반복문**으로 모든 상품의 가격을 합산해야 합니다.
2. 마지막으로 **조건문**을 통해 무료 배송 여부를 판단하고, 이 모든 과정을 하나의 **함수**로 묶어 완성할 수 있습니다.

- 하나의 기능을 만드는 데 필요한 “데이터”, “연산자”, “반복문”, “조건문”, “함수” 등 자바 스크립트의 가장 기본적인 문법을 학습합니다.

학습 목표

- ✓ 원시 자료형과 참조 자료형의 차이점을 설명할 수 있다.
- ✓ 동등(==) 연산자와 일치(===) 연산자의 차이를 안다.
- ✓ for...in과 for...of 반복문의 차이와 용도를 안다.
- ✓ 함수 선언식과 표현식의 차이를 알고 함수를 정의한다.
- ✓ 화살표 함수 표현식을 사용하여 간결하게 함수를 정의한다.
- ✓ 나머지 매개변수와 전개 구문을 함수에 활용할 수 있다.
- ✓ 템플릿 리터럴을 사용해 변수를 포함한 문자열을 만든다.

데이터 타입

원시 자료형(Primitive type)

- 값(value) 자체가 변수에 직접 저장되는 자료형
- 불변(immutable)이며, 변수 간 할당 시 값이 복사
- Number, String, Boolean, null, undefined

참조 자료형(Reference type)

- 데이터가 저장된 메모리의 주소가 변수에 저장되는 자료형
- 가변(Mutable)이며, 변수 간 할당 시 주소가 복사
- Objects(Object, Array, Function)

TIP

- 참조 자료형은 주소를 복사하므로, 복사본을 수정하면 원본의 값도 함께 변경될 수 있어 주의해야 합니다.
- 값이 불변이라는 것은, 값의 일부를 직접 수정할 수 없다는 의미입니다.
(let str = 'ssafy'; str[0] = 'S';) 처럼 할당하는 것은 불가능

원시 자료형 예시

- 변수에 할당될 때 값이 복사됨
 - 변수 간에 서로 영향을 미치지 않음

```
const a = 'bar'
console.log(a) // bar

a.toUpperCase()
console.log(a) // bar
```

```
let a = 10
let b = a

b = 20

console.log(a) // 10
console.log(b) // 20
```


참조 자료형 예시

- 객체를 생성하면 객체의 메모리 주소를 변수에 할당
 - 변수 간에 서로 영향을 미침

```
const obj1 = { name: 'Alice', age: 30 }  
const obj2 = obj1  
  
obj2.age = 40  
  
console.log(obj1.age) // 40  
console.log(obj2.age) // 40
```

원시 자료형

원시 자료형 종류

- Number
- String
- null
- undefined
- Boolean

원시 자료형 - Number

Number

정수 또는 실수형 숫자를 표현하는 자료형

- 사칙연산 및 나머지 연산 가능
- 문자열과 + 연산 시, 숫자가 문자열로 자동 형 변환되어 연결
- 정수와 실수 구분이 없고, 모든 숫자를 단일 타입으로 처리

```
const a = 13
const b = -5
const c = 3.14
const d = 2.998e8 // 2.998 * 10^8 = 299,800,000
const e = Infinity
const f = -Infinity
const g = NaN // Not a Number를 나타내는 값
```


원시 자료형 - String

String

| 텍스트 데이터를 표현하는 자료형

- '+' 연산자를 사용해 문자열끼리 결합
- 백셈, 곱셈, 나눗셈 불가능

```
const firstName = 'Tony'  
const lastName = 'Stark'  
const fullName = firstName + lastName  
  
console.log(fullName) // TonyStark
```

Template literals (템플릿 리터럴)

- 내장된 표현식을 허용하는 향상된 문자열 작성 방식
- Backtick(` `)을 이용하며, 여러 줄에 걸쳐 문자열을 정의할 수도 있고 JavaScript의 변수를 문자열 안에 바로 연결할 수 있음
- 표현식은 '\$'와 중괄호({expression}) 로 표기
- ES6+ 부터 지원

```
const age = 100
const message = `홍길동은 ${age}세입니다.`
console.log(message) // 홍길동은 100세입니다.
```

null

- 프로그래머가 의도적으로
'값이 없음'을 나타낼 때 사용

```
let a = null  
console.log(a) // null
```

undefined

- 시스템이나 JavaScript 엔진이
'값이 할당되지 않음'을 나타낼 때 사용

```
let b  
console.log(b) // undefined
```

TIP

- 'null'의 타입은 'object', 'undefined'의 타입은 'undefined' 이므로 타입 검사 시 주의하세요.
- return 이 없는 함수나, 인자가 전달되지 않은 매개변수는 기본적으로 undefined가 할당됩니다.
- 산술 연산 시 'null' 은 0으로 취급되지만, 'undefined'는 계산 불가능한 NaN 값을 만듭니다.

원시 자료형 - Boolean

Boolean

참과 거짓을 나타내는 논리적인 자료형

- 조건문 또는 반복문에서 Boolean이 아닌 데이터 타입은
- “자동 형변환 규칙”에 따라 true 또는 false로 변환됨

자동 형변환

데이터 타입	false	true
undefined	항상 false	X
null	항상 false	X
Number	0, -0, NaN	나머지 모든 경우
String	"" (빈 문자열)	나머지 모든 경우

<표1_Javascript_Basic syntax 01_자동 형변환>

연산자

연산자 종류

- 할당 연산자
- 증가 & 감소 연산자
- 비교 연산자
- 동등 연산자
- 일치 연산자
- 논리 연산자

할당 연산자

- 오른쪽에 있는 피연산자의 평가 결과를 왼쪽 피연산자에 할당하는 연산자
- 단축 연산자 지원

```
let a = 0

a += 10
console.log(a) // 10

a -= 3
console.log(a) // 7

a *= 10
console.log(a) // 70

a %= 7
console.log(a) // 0
```



단축 연산자 :

$x = x + 1$ 을 $x += 1$ 처럼
코드를 짧게 줄여주는 연산자

<개념1_Javascript_Basic syntax 01_단축 연산자>

증가 & 감소 연산자

- 증가 연산자 ('++')

- 피연산자를 증가(1을 더함)시키고 연산자의 위치에 따라 증가하기 전이나 후의 값을 반환

- 감소 연산자 ('--')

- 피연산자를 감소(1을 뺌)시키고 연산자의 위치에 따라 감소하기 전이나 후의 값을 반환

➤ 코드의 가독성을 위해 `a += 1`, `a -= 1`과 같이 더 명시적인 표현을 권장

```
let x = 3
const y = x++
console.log(x, y) // 4 3
```

```
let a = 3
const b = ++a
console.log(a, b) // 4 4
```

비교 연산자

- 피연산자들(숫자, 문자, Boolean 등)을 비교하고 결과 값을 boolean으로 반환하는 연산자

```
3 > 2 // true
3 < 2 // false

'A' < 'B' // true
'Z' < 'a' // true
'가' < '나' // true
```


동등 연산자 (==)

- 두 피연산자가 같은 값으로 평가되는지 비교한 후 boolean 값을 반환
- '암묵적 타입 변환' 통해 타입을 일치시킨 후 같은 값인지 비교
- 두 피연산자가 모두 객체일 경우 메모리의 같은 객체를 바라보는지 판별

```
console.log(1 == 1) // true  
console.log('hello' == 'hello') // true  
console.log('1' == 1) // true  
console.log(0 == false) // true
```

TIP

- `0 == false`, `'' == []` 가 true 가 되는 등, 직관과 다른 암묵적 타입 변환이 일어나니 주의하세요.
- 내용이 같아도 다른 객체이면 false 입니다. `[1] == [1]` 은 false 이니 주의하세요.

일치 연산자 (===)

- 두 피연산자의 값과 타입이 모두 같은 경우 true를 반환
- 같은 객체를 가리키거나, 같은 타입이면서 같은 값인지를 비교
- 엄격한 비교가 이뤄지며 암묵적 타입 변환이 발생하지 않음
- 특별한 경우를 제외하고는, 예측하지 못한 결과를 방지하기 위해 **일치 연산자(===) 사용을 권장**

```
console.log(1 === 1) // true
console.log('hello' === 'hello') // true
console.log('1' === 1) // false
console.log(0 === false) // false
```

논리 연산자

- and 연산
 - &&
- or 연산
 - ||
- not 연산
 - !
- 단축 평가 지원

```
true && false // false
true && true  // true
```

```
false || true // true
false || false // false
```

```
!true // false
```

```
1 && 0 // 0
```

```
0 && 1 // 0
```

```
4 && 7 // 7
```

```
1 || 0 // 1
```

```
0 || 1 // 1
```

```
4 || 7 // 4
```

조건문

if

조건 표현식의 결과값을 boolean 타입으로 변환 후
참/거짓을 판단

```
<!-- if-statement.html -->

const name = 'customer'
if (name === 'admin') {
  console.log('관리자님 환영해요')
} else if (name === 'customer') {
  console.log('고객님 환영해요')
} else {
  console.log(`반갑습니다. ${name}님`)
}
```


삼항 연산자

- 간단한 조건부 로직을 간결하게 표현할 때 유용
- 복잡한 로직이나 대다수의 경우에는 가독성이 떨어질 수 있으므로 적절한 상황에서만 사용할 것
- condition
 - 평가할 조건 (true 또는 false로 평가)
- expression1
 - 조건이 true일 경우 반환할 값 또는 표현식
- expression2
 - 조건이 false일 경우 반환할 값 또는 표현식



삼항 연산자 표현식 :
condition ? expression1 : expression2

<개념2_Javascript_Basic syntax 01_삼항 연산자 표현식>

```
const age = 20
const message = (age >= 18) ? '성인' : '미성년자'
console.log(message) // '성인'
```

반복문

반복문 종류

- while
- for
- for ... in
- for ... of

while 반복문

while

조건문이 참이면 문장을 계속해서 수행

```
while (조건문) {  
  // do something  
}
```

```
<!-- loops-and-iteration.html -->  
  
let i = 0  
while (i < 6) {  
  console.log(i)  
  i += 1  
}
```

for 반복문

for

특정한 조건이 거짓으로 판별될 때까지 반복

```
for ([초기문]; [조건문]; [증감문]) {  
    // do something  
}
```

```
<!-- loops-and-iteration.html -->  
  
for (let i = 0; i < 6; i++) {  
    console.log(i)  
}
```


for 동작 원리

```
for (let i = 0; i < 6; i++) {  
  console.log(i)  
}
```

// 1. 반복문 진입 및 변수 i 선언

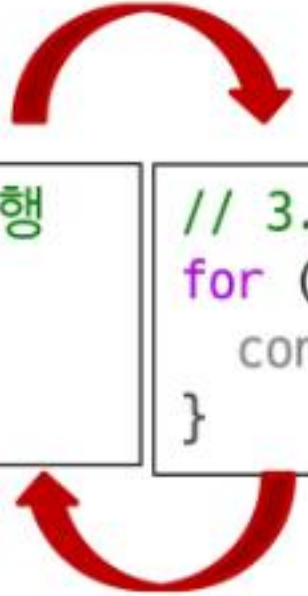
```
for (let i = 0; i < 6; i++) {  
  console.log(i)  
}
```

// 2. 조건문 평가 후 코드 블록 실행

```
for (let i = 0; i < 6; i++) {  
  console.log(i) // 0  
}
```

// 3. 코드 블록 실행 이후 i 값 증가

```
for (let i = 0; i < 6; i++) {  
  console.log(i)  
}
```



for ... in

for ... in 반복문

for ... in

객체의 열거 가능한(enumerable)
속성(property)의 **키(key)**에 대해 반복

```
for (variable in object) {  
  statement  
}
```

```
<!-- loops-and-iteration.html -->  
  
const fruits = { a: 'apple', b: 'banana' }  
for (const property in object) {  
  console.log(property) // a, b  
  console.log(object[property]) // apple, banana  
}
```

for ... of

for ... of 반복문

for ... of

반복 가능한(iterable) 객체(배열, 문자열 등)의
값(value)에 대해 반복합니다.

```
for (variable in iterable) {  
  statement  
}
```

```
<!-- loops-and-iteration.html -->  
  
const numbers = [0, 1, 2, 3]  
for (const number of numbers) {  
  console.log(number) // 0, 1, 2, 3  
}
```


for ... in 과학 for ... of

for ... in

- 배열과 객체

```
// Array
const arr = ['a', 'b', 'c']
for (const elem in arr) {
  console.log(elem) // 0 1 2
}
```

```
// Object
const capitals = {
  korea: '서울',
  japan: '도쿄',
  china: '베이징',
}
for (const capital in capitals) {
  console.log(capital) // korea japan china
}
```

for ... of

- 배열과 객체

```
// Array
const arr = ['a', 'b', 'c']
for (const elem of arr) {
  console.log(elem) // a b c
}
```

```
// Object
const capitals = {
  korea: '서울',
  japan: '도쿄',
  china: '베이징',
}
for (const capital of capitals) {
  console.log(capital)
  // TypeError: capitals is not iterable
}
```

배열 반복과 for...in (1/2)

- 객체의 관점에서 보면, 배열의 인덱스도 "정수 형태의 이름을 가진 열거 가능한 속성"
- for...in은 정수가 아닌 이름과 속성을 포함하여 열거 가능한 모든 속성을 반환
- 내부적으로 for...in은 배열의 반복자가 아닌 속성 열거를 사용하기 때문에 특정 순서에 따라 인덱스를 반환하는 것을 보장할 수 없음

➤따라서 for...in은 인덱스의 순서가 중요한 배열에서는 사용하지 않음

➤배열에서는 **for 문, for...of**를 사용

배열 반복과 for...in (2/2)

- 객체 관점에서 배열의 인덱스는 정수 이름을 가진 속성이기 때문에 인덱스가 출력됨 (순서 보장 X)

```
const arr = ['a', 'b', 'c']
for (const i in arr) {
  console.log(i) // 0, 1, 2
}
for (const i of arr) {
  console.log(i) // a, b, c
}
```

TIP

- for ... in 은 배열의 값이 아닌 인덱스를, 그것도 숫자가 아닌 '문자열'로 반환하니 주의해야 합니다.
- 배열의 프로토타입에 추가된 속성까지 순회할 수 있어, 예기치 않은 버그를 유발할 수 있습니다.

반복문 사용 시 const 사용 여부

- for 문
 - for (let i = 0; i < arr.length; i++) { ... }의 경우에는 최초 정의한 i 를 “재할당” 하면서 사용하기 때문에 **const를 사용하면 에러 발생**
- for...in , for...of
 - 재할당이 아니라, 매 반복마다 다른 속성 이름이 변수에 지정되는 것이므로 **const를 사용해도 에러가 발생하지 않음**
 - 단, const 특징에 따라 블록 내부에서 변수를 수정할 수 없음

반복문 종합

키워드	특징	스코프
while	.	블록 스코프
for	.	
for ... in	object 순회	
for ... of	iterable 순회	

<표2_Javascript_Basic syntax 01_반복문 종합>

함수

Function

- 참조 자료형에 속하며 모든 함수는 Function object

원시 자료형(Primitive type)

- 변수에 값이 직접 저장되는 자료형
- 불변이며, 값이 복사 됨
- Number, String, Boolean, null, undefined

참조 자료형(Reference type)

- 객체의 주소가 저장되는 자료형
- 가변이며, 주소가 복사 됨
- Objects(Object, Array, Function)

함수 정의

함수 구조

```
function name ([param[, param[, ..., param]]) {  
  statements  
  return value  
}
```

- function 키워드
- 함수의 이름
- 함수의 매개변수
- 함수의 body를 구성하는 statements
- ❖ return 문이 없거나 return 뒤에 값이 없으면, 함수는 undefined를 반환합니다.

함수 정의 2가지 방법 (1/2)

선언식 (function declaration)

```
function funcName () {  
  statements  
}
```

표현식(function expression)

```
const funcName = function () {  
  statements  
}
```

함수 정의 2가지 방법 (2/2)

선언식 (function declaration)

```
function add (num1, num2) {  
  return num1 + num2  
}  
add(1, 2) // 3
```

표현식(function expression)

```
const sub = function (num1, num2) {  
  return num1 - num2  
}  
sub(2, 1) // 1
```

함수 선언식 특징

- 호이스팅 됨
- 코드의 구조와 가독성 면에서는 표현식에 비해 장점이 있음

```
add(1, 2) // 3

function add (num1, num2) {
  return num1 + num2
}
```

TIP

- 호이스팅
 - 자바스크립트 엔진이 코드 실행 전 변수와 함수 선언을 스코프 최상단으로 끌어올리는 동작 방식
 - 선언식으로 된 함수는 선언 전에 호출하는 것이 가능해 짐

함수 표현식 특징

- 호이스팅 되지 않음
 - 변수 선언만 호이스팅되고 함수 할당은 실행 시점에 이루어짐
- 함수 이름이 없는 '익명 함수'를 사용할 수 있음



익명 함수:
이름 없이, 필요할 때 즉시
만들어서 사용하는 일회용 함수

<개념3_Javascript_Basic syntax 01_익명 함수>

```
sub(2, 1) // ReferenceError: Cannot access 'sub' before initialization
```

```
const sub = function (num1, num2) {  
  return num1 - num2  
}
```

함수 표현식 사용을 권장하는 이유

- 예측 가능성
 - 호이스팅의 영향을 받지 않아 코드 실행 흐름을 더 명확하게 예측할 수 있음
- 유연성
 - 변수에 할당되므로 함수를 값으로 다루기 쉬움
- 스코프 관리
 - 블록 스코프를 가지는 `let`이나 `const`와 함께 사용하여 더 엄격한 스코프 관리가 가능

매개변수

매개변수 정의 방법

1. 기본 함수 매개 변수

2. 나머지 매개 변수



매개 변수:

함수가 외부로부터 값을 전달받기 위해 만들어 놓은 변수

<개념4_Javascript_Basic syntax 01_매개 변수>

1. 기본 함수 매개변수 (Default function parameter)

- 함수 호출 시 인자를 전달하지 않거나 undefined를 전달할 경우, 지정된 기본값으로 매개변수를 초기화

```
const greeting = function (name = 'Anonymous') {  
  return `Hi ${name}`  
}  
  
greeting() // Hi Anonymous
```

2. 나머지 매개변수 (Rest parameter)

- 정해지지 않은 개수의 인자들을 배열로 모아서 받는 방법
- 작성 규칙
 - 함수 정의 시 나머지 매개변수는 하나만 작성할 수 있음
 - 나머지 매개변수는 함수 정의에서 매개변수 마지막에 위치해야 함

```
const myFunc = function (param1, param2, ...restPrms) {  
  return [param1, param2, restPrms]  
}
```

```
myFunc(1, 2, 3, 4, 5) // [1, 2, [3, 4, 5]]  
myFunc(1, 2) // [1, 2, []]
```

매개변수와 인자 개수가 불일치 할 때 (1/2)

- 매개변수 개수 > 인자 개수
 - 누락된 인자는 undefined로 할당

```
const threeArgs = function (param1, param2, param3) {  
  return [param1, param2, param3]  
}
```

```
threeArgs() // [undefined, undefined, undefined]  
threeArgs(1) // [1, undefined, undefined]  
threeArgs(2, 3) // [2, 3, undefined]
```


매개변수와 인자 개수가 불일치 할 때 (2/2)

- 매개변수 개수 < 인자 개수
 - 초과 입력한 인자는 사용하지 않음

```
const noArgs = function () {  
  return 0  
}  
noArgs(1, 2, 3) // 0  
  
const twoArgs = function (param1, param2) {  
  return [param1, param2]  
}  
twoArgs(1, 2, 3) // [1, 2]
```

Spread syntax

Spread syntax '...'

- 전개 구문
- 배열이나 문자열처럼 반복 가능한(iterable) 항목들을 개별 요소로 펼치는 것
- 전개 대상에 따라 역할이 다름
 - 배열이나 객체의 요소를 개별적인 값으로 분리하거나 다른 배열이나 객체의 요소를 현재 배열이나 객체에 추가하는 등

전개 구문 활용처

1. 함수와의 사용
 - ① 함수 호출 시 인자 확장
 - ② 나머지 매개변수 (압축)
2. 객체와의 사용 (객체 파트에서 진행)
3. 배열과의 활용 (배열 파트에서 진행)

전개 구문 활용 (1/2)

- 함수와의 사용

- 인자 확장 (함수 호출 시)

```
<!-- spread-syntax.html -->
```

```
function myFunc(x, y, z) {  
  return x + y + z  
}
```

```
let numbers = [1, 2, 3]
```

```
console.log(myFunc(...numbers)) // 6
```

전개 구문 활용 (2/2)

- 함수와의 사용

- 2. 나머지 매개변수 (압축)

```
<!-- spread-syntax.html -->
```

```
function myFunc2(x, y, ...restArgs) {  
  return [x, y, restArgs]  
}
```

```
console.log(myFunc2(1, 2, 3, 4, 5)) // [1, 2, [3, 4, 5]]  
console.log(myFunc2(1, 2)) // [1, 2, []]
```


화살표 함수 표현식

화살표 함수 표현식

화살표 함수 표현식

Arrow function expressions

함수 표현식의 간결한 표현법

```
const arrow = function (name) {  
  return `hello, ${name}`  
}
```



```
const arrow = name => `hello, ${name}`
```

화살표 함수 작성 과정 (1/3)

1. function 키워드 제거 후 매개변수와 중괄호 사이에 화살표(=>) 작성

```
const arrow1 = function (name) {  
  return `hello, ${name}`  
}
```

// 1. function 키워드 삭제 후 화살표 작성

```
const arrow2 = (name) => { return `hello, ${name}` }
```

화살표 함수 작성 과정 (2/3)

1. function 키워드 제거 후 매개변수와 중괄호 사이에 화살표(=>) 작성
2. 함수의 매개변수가 하나 뿐이라면, 매개변수의 `()` 제거 가능
(단, 생략하지 않는 것을 권장)

```
const arrow1 = function (name) {  
  return `hello, ${name}`  
}  
  
// 1. function 키워드 삭제 후 화살표 작성  
const arrow2 = (name) => { return `hello, ${name}` }  
  
// 2. 인자의 소괄호 삭제 (인자가 1개일 경우에만 가능)  
const arrow3 = name => { return `hello, ${name}` }
```


화살표 함수 작성 과정 (3/3)

1. function 키워드 제거 후 매개변수와 중괄호 사이에 화살표(=>) 작성
2. 함수의 매개변수가 하나 뿐이라면, 매개변수의 `()` 제거 가능
(단, 생략하지 않는 것을 권장)
3. 함수 본문의 표현식이 한 줄이라면, `{}` 와 `return` 제거 가능

```
const arrow1 = function (name) {  
  return `hello, ${name}`  
}  
  
// 1. function 키워드 삭제 후 화살표 작성  
const arrow2 = (name) => { return `hello, ${name}` }  
  
// 2. 인자의 소괄호 삭제 (인자가 1개일 경우에만 가능)  
const arrow3 = name => { return `hello, ${name}` }  
  
// 3. 중괄호와 return 삭제 (함수 본문이 return을 포함한 표현식 1개일 경우에만 가능)  
const arrow4 = name => `hello, ${name}`
```


참고

NaN 예시

NaN을 반환하는 경우 예시

1. 숫자로서 읽을 수 없음 (`Number(undefined)`)
2. 결과가 허수인 수학 계산식 (`Math.sqrt(-1)`)
3. 피연산자가 NaN (`7 ** NaN`)
4. 정의할 수 없는 계산식 (`0 * Infinity`)
5. 문자열을 포함하면서 덧셈이 아닌 계산식 (`'가' / 3`)

TIP

- NaN은 자기 자신과 비교해도 False가 나오는 유일한 값입니다. (`NaN === NaN // False`)
- NaN은 '숫자가 아님'이라는 뜻이지만, 타입을 확인하면 'number'가 반환됩니다.

null & undefined

‘값이 없음’에 대한 표현이 null과 undefined 2가지인 이유 (1/2)

1. 역사적 맥락

- JavaScript가 처음 만들어질 때, null은 '객체가 없음'을 나타내기 위해 도입
- undefined는 나중에 추가되어 '값이 할당되지 않음'을 나타내게 됨

2. typeof null의 결과가 "object"인 이유 (역사적 배경)

- 초기 버전에서 값의 타입을 나타내는 데 32비트 시스템을 사용
- 타입 태그로 하위 3비트를 사용했는데, '000'은 객체를 나타냄
- null은 모든 비트가 0인 특별한 값(NULL 포인터)으로 표현되었는데,
하필 객체 타입을 나타내는 태그 또한 000이었기 때문에 null을 객체로 잘못 판별하게 됨

```
typeof null      // "object"  
typeof undefined // "undefined"
```


‘값이 없음’에 대한 표현이 null과 undefined 2가지인 이유 (2/2)

3. ECMAScript의 표준화

- ECMAScript 명세에서는 null을 원시 자료형으로 정의
- 그러나 typeof null의 결과는 역사적인 이유로 "object"를 유지
- ECMAScript 5 개발 중 이 문제를 수정하려는 시도가 있었지만, 이미 typeof null === 'object'라는 전제하에 만들어진 수많은 웹 사이트와의 하위 호환성 문제 때문에 수정되지 못함

```
typeof null           // "object"  
typeof undefined     // "undefined"  
  
null == undefined    // true  
null === undefined   // false
```

화살표 함수 심화

화살표 함수 심화

// 1. 인자가 없다면 () or _ 로 표시 가능

```
const noArgs1 = () => 'No args'
```

```
const noArgs2 = _ => 'No args'
```

// 2-1. object를 return 한다면 return 을 명시적으로 작성해야 함

```
const returnObject1 = () => { return { key: 'value' } }
```

// 2-2. return을 작성하지 않으려면 객체를 소괄호로 감싸야 함

```
const returnObject2 = () => ({ key: 'value' })
```

연산자

- 2963. 다양한 연산자
- 3076. 연산자와 제어문

조건문

- 2960. 기초 문법 디버깅 - 조건문과 변수 선언
- 2965. 제어문

반복문

- 2964. 배열 순회
- 3077. 배열과 객체 순회
- 3078. 반복문과 제어문
- 2961. 기초 문법 디버깅 - 반복문

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 중 원시 자료형(Primitive type)이 아닌 것은?

- a) Number
- b) String
- c) Array
- d) undefined

2 null과 undefined에 대한 설명으로 틀린 것은?

- a) null은 의도적인 '값 없음'을 의미한다.
- b) undefined는 값이 할당되지 않았음을 의미한다.
- c) typeof null의 결과는 'null'이다.
- d) null == undefined는 true이다.

3 ==와 === 연산자의 주된 차이점은?

- a) 연산 속도
- b) 타입 변환 여부
- c) 반환 값의 타입
- d) 객체 비교 가능 여부

4 논리 연산자 중 단축 평가를 활용하는 코드는?

- a) true && false
- b) 0 && 3
- c) 4 & 7
- d) !true

Q 각 문제에 올바른 답안을 생각해봅시다

5 삼항 연산자에 대한 설명으로 올바른 것은?

- a) 세 개의 피연산자를 가진다.
- b) 항상 if문보다 가독성이 좋다.
- c) 조건이 거짓일 때 첫 번째 표현식을 반환한다.
- d) 반복문 내에서는 사용할 수 없다.

6 객체의 '키(key)'를 순회하기에 가장 적합한 반복문은?

- a) for
- b) while
- c) for...in
- d) for...of

7 배열의 '값(value)'을 순회하는 반복문은?

- a) for
- b) while
- c) for...in
- d) for...of

8 함수 선언식과 함수 표현식의 가장 큰 차이점은?

- a) 매개변수 개수
- b) 반환 값 존재 여부
- c) 호이스팅 발생 여부
- d) 익명 함수 사용 가능 여부

Q 각 문제에 올바른 답안을 생각해봅시다

9 임의의 수의 인자를 배열로 받는 매개변수 문법은?

- a) 기본 매개변수
- b) 가변 매개변수
- c) 나머지 매개변수
- d) 전개 구문

10 화살표 함수로 옳게 축약된 것은?

- a) `const arrow = name => { 'hello, ' + name }`
- b) `const arrow = name => 'hello, ' + name`
- c) `const arrow = (name) => return 'hello, ' + name`
- d) `const arrow = => 'hello, ' + name`

확인 문제



정답 및 해설

1. c) Array 2. c) typeof null의 결과는 'null'이다. 3. b) 타입 변환 여부 4. b) 0 && 3 5. a) 세 개의 피연산자를 가진다.
6. c) for...in 7. d) for...of 8. c) 호이스팅 발생 여부 9. c) 나머지 매개변수 10. b) const arrow = name => 'hello, ' + name

1. Array는 객체(Object)에 속하는 참조 자료형이며, 원시 자료형이 아닙니다.
2. JavaScript 초기 버전의 오류로 인해 typeof null은 'object'를 반환합니다.
3. ==는 타입을 변환하여 비교하지만, ===는 타입까지 엄격하게 비교합니다.
4. && 연산자는 첫 피연산자가 참일 경우, 뒤를 쳐다보지 않고 그대로 중지합니다.
5. 조건 ? 참일때_표현식 : 거짓일때_표현식 구조로 3개의 피연산자를 사용합니다.

6. for...in은 객체의 열거 가능한 속성(키)에 대해 반복하는 데 사용됩니다.
7. for...of는 배열과 같은 반복 가능한 객체의 각 요소(값)를 순회하는 데 사용됩니다.
8. 함수 선언식은 호이스팅되어 선언 전에 호출 가능하지만, 함수 표현식은 그렇지 않습니다.
9. 나머지 매개변수(...rest)는 정해지지 않은 수의 인자들을 배열로 모아줍니다.
10. 본문이 한 줄의 표현식이면 중괄호와 return을 동시에 생략할 수 있습니다.

핵심 키워드

개념	설명	예시
원시 자료형	값이 직접 저장되는 불변 데이터	Number, String, Boolean 등
일치 연산자 (===)	값과 타입이 모두 같은지 비교	'1' === 1 // false
for...of 반복문	반복 가능한 객체의 값을 반복	for (const i of arr) { ... }
함수 표현식	변수에 함수를 할당하여 정의	const sub = function () {}
화살표 함수	함수 표현식의 간결한 표현법	const arrow = () => {}
나머지 매개변수	여러 인자를 배열로 받는 매개변수	function (...args) {}
전개 구문 (...)	반복 가능한 항목을 펼치는 문법	myFunc(...numbers)

〈표3_Javascript_Basic syntax 01_핵심키워드〉

활동 정리

오늘 공부한 내용 요약 및 정리

- JavaScript의 데이터 타입은 크게 두 가지(원시 자료형, 참조 자료형)
- 원시 자료형 (Primitive type)
 - 변수에 값이 직접 저장되는 자료형
 - Number, String, Boolean, null, undefined가 여기에 해당
 - 데이터가 불변(immutable)이며, 다른 변수에 할당할 때 값이 복사
- 참조 자료형 (Reference type)
 - 객체의 메모리 주소가 변수에 저장되는 자료형
 - Object, Array, Function 등이 여기에 해당
 - 데이터가 가변(mutable)이며, 다른 변수에 할당할 때 주소가 복사되어 원본과 사본이 서로 영향을 줌

활동 정리

오늘 공부한 내용 요약 및 정리

• 원시 자료형의 종류

- Number: 정수와 실수를 모두 표현하는 숫자 자료형
- String: 텍스트 데이터를 나타내며, + 연산자로 문자열을 결합 가능
- 템플릿 리터럴: 백틱(`)을 사용해 변수를 \${} 형태로 문자열에 쉽게 삽입할 수 있는 방식
- null & undefined
 - null은 개발자가 의도적으로 '값이 없음'을 나타낼 때 사용
 - undefined는 '값이 할당되지 않음'을 시스템이 나타낼 때 사용
- Boolean: true와 false를 나타내는 논리 자료형

활동 정리

오늘 공부한 내용 요약 및 정리

• 비교 연산자

- 동등 연산자 (==)
 - 두 피연산자의 타입을 일치시킨 후 값을 비교
 - 이 과정에서 예측하기 어려운 결과가 나올 수 있어 사용 시 주의가 필요

• 일치 연산자 (===)

- 두 피연산자의 값과 타입을 모두 비교하며, 암묵적 타입 변환을 하지 않음
- 더 엄격하고 예측 가능하므로 이 연산자의 사용이 권장

• 논리 연산자

- &&(AND), ||(OR), !(NOT)가 있으며, 단축 평가를 지원

활동 정리

오늘 공부한 내용 요약 및 정리

- 반복문

- for...in:

- 객체의 키(key)를 순회, 배열에는 부적합
 - 배열에 사용하면 인덱스를 문자열로 반환하고 순서를 보장하지 않아 권장되지 않음

- for...of

- 배열 등 반복 가능한 객체의 값(value)을 순회
 - 객체는 반복 가능하지 않으므로 직접 사용할 수 없음

활동 정리

오늘 공부한 내용

요약

및

정리

• 함수

• 함수 정의 방법:

• 함수 선언식

- `function funcName() { }` 형태로 정의하며, 호이스팅(hoisting)의 영향을 받음

• 함수 표현식

- `const funcName = function() { }` 형태로 변수에 함수를 할당
- 호이스팅되지 않아 코드의 예측 가능성이 높아져 사용이 권장

• 화살표 함수

- `const arrow = () => { }` 형태로 함수 표현식을 더 간결하게 작성하는 방법
- 함수 본문이 한 줄이면 `}`와 `return`을 생략할 수 있음

활동 정리

오늘 공부한 내용

요약

및

정리

- 함수

- 매개변수

- 기본 매개변수: `function (name = 'Anonymous')`처럼 인자가 전달되지 않을 경우를 대비해 기본값을 설정
 - 나머지 매개변수: `function (...args) {}` 형태로 작성하며, 정해지지 않은 수의 인자들을 배열로 받을 수 있음
 - 전개 구문 (...): `myFunc(...numbers)`처럼 배열이나 반복 가능한 객체를 펼쳐서 함수의 인자로 전달할 때 사용

활동 정리

장바구니 총액을 계산하고, 특정 금액이 넘으면 '무료 배송'을 적용하는 기능, 어떻게 만들까요?

```
const calculateCartTotal = function (items) {  
  let total = 0;  
  
  for (const item of items) {  
    total += item.price * item.quantity;  
  }  
  
  if (total >= 50000) {  
    console.log(`총액: ${total}원 (무료 배송)`);  
  } else {  
    console.log(`총액: ${total}원`);  
  }  
};
```

←

감사합니다

→

